الجامعة الإسلامية العالمية ماليزيا
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
يونيورسيتي إسلام انتارابغسا ملايسيا

(Company No.: 198301005860)

Garden of Knowledge and Virtue

MECHATRONICS SYSTEM INTEGRATION : MCTA 3203

LABORATORY REPORT

SECTION 1

TITLE : Bluetooth Data Interfacing (WEEK 8)

LECTURER'S NAME: ASSOC. PROF. IR. TS. DR. ZULKIFLI BIN ZAINAL ABIDIN

DATE OF EXPERIMENT : 3 DECEMBER 2025

DATE OF SUBMISSION : 10 DECEMBER 2025

GROUP : 4

NO.	NAME	MATRIC NO.
1	MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	2311779
2	AHMAD HARITH IMRAN BIN MOHD YUSOF	2311581
3	ARIFA AQILAH BINTI ABDUL HALIM	2311530

ACKNOWLEDGEMENTS

We would like to express our heartfelt appreciation to Dr. Zulkifli Bin Zainal Abidin for his continuous guidance, support, and insightful explanations throughout this experiment. His expertise greatly helped us understand the concepts of interfacing and digital control using Arduino. We would also like to thank our lab assistant for providing technical assistance and ensuring that the experiment ran smoothly. Finally, we extend our gratitude to our classmates for their cooperation and teamwork during the lab session.

ABSTRACT

This experiment presents the development of a wireless temperature monitoring and control system using Bluetooth communication. The main goal was to establish a stable link between an ESP32 or Arduino board equipped with a DHT11 temperature sensor and a paired device such as a computer or smartphone. The microcontroller continuously captured real-time temperature data from the sensor and transmitted it wirelessly through a Bluetooth serial connection, either via an HC-05 module or the ESP32's built-in Bluetooth capability. A receiving device, using a serial terminal application or a Python script, displayed the incoming temperature readings. In addition to monitoring, the system was programmed to interpret simple remote commands like "FAN ON" or "FAN OFF," demonstrating its ability to support basic control functions. Overall, the experiment successfully confirmed reliable two-way communication, effective temperature data transmission, and responsive remote command processing. This work showcases a fundamental yet practical approach to integrating wireless communication into mechatronic systems for remote supervision and control.

TABLE OF CONTENT

1.0 INTRODUCTION

This experiment explores mechatronics system integration by developing a wireless data interface for remote sensing and control. The main objective is to build a functional Bluetooth-based temperature monitoring system using an ESP32 development board or an Arduino paired with an HC-05 Bluetooth module. The system is designed to read temperature values from a DHT11 sensor and transmit them wirelessly to a computer or smartphone. In addition, it demonstrates two-way communication by allowing the paired device to send simple control commands, such as “FAN ON” or “FAN OFF,” which the microcontroller interprets to simulate basic environment control. An LED connected to the microcontroller is used to visually indicate the activation or deactivation of the simulated device, providing a clear demonstration of command response.

The experiment is grounded in core concepts of embedded systems, digital temperature sensing, and wireless serial communication. The ESP32 or Arduino acts as the central embedded controller, receiving temperature data from the DHT11 sensor through a single-wire digital interface. Bluetooth serves as the wireless communication channel, functioning as a virtual serial port to stream data to a remote terminal application or Python script. When using an Arduino, the SoftwareSerial library enables Bluetooth communication on additional pins while keeping the hardware serial port available for debugging.

The experiment hypothesizes that a stable, two-way Bluetooth link can be established to provide real-time temperature monitoring and receive control commands. Expected outcomes include successful device pairing, continuous display of temperature data on the remote terminal, proper microcontroller response to incoming control commands indicated by the LED, and the ability to analyze the logged temperature data for observable trends.

2.0 MATERIAL AND EQUIPMENT

1. ESP32 development board
2. Temperature sensor (DHT11)
3. Smartphone or computer with Bluetooth support
4. Power supply for the ESP32
5. Breadboard
6. Jumper wires

3.0 EXPERIMENTAL SETUP

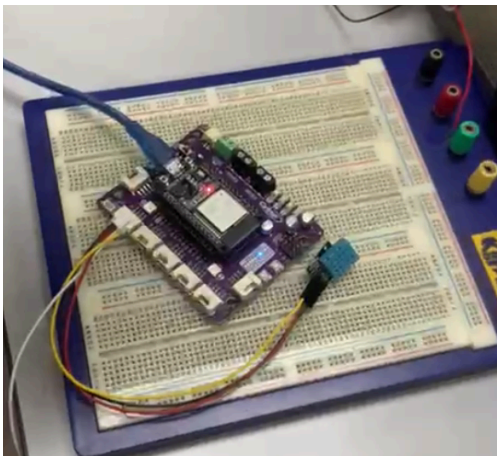


Figure A: Components connected to ESP32

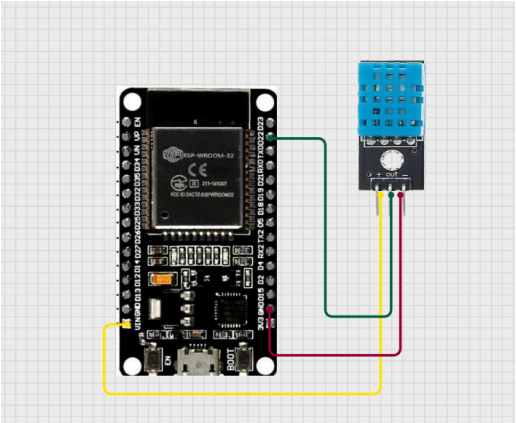


Figure B: Components connected to ESP32 via Cirkit Designer

1. The pin is connected as below;
    - DHT 11 sensor connected to pin 22
    - LED on the ESP 32 uses pin 12
2. Run the Arduino IDE program
3. Simulate the program using the coding

4.0 METHODOLOGY

1. The ESP32 circuit was assembled according to the required wiring layout, using only the ESP32's built-in Bluetooth capability.
2. The DHT11 temperature sensor was connected to Digital Pin 22, and the LED (acting as a simulated actuator) was connected to Digital Pin 12.
3. The ESP32 program was uploaded, containing the Bluetooth communication setup, temperature-reading logic, and control command handling.
4. The ESP32 system was powered on, and Bluetooth pairing was performed using a smartphone or computer, connecting to the device named "Group4."
5. After pairing, the user opened a serial terminal to send commands and observe the incoming temperature and humidity data.
6. The command 'O' was sent to turn the simulated actuator ON, and 'F' was sent to turn it OFF.
7. Temperature and humidity readings were monitored every 2 seconds, confirming correct data transmission and LED response to the control commands.
8. The process was repeated several times to verify stable two-way communication and consistent temperature reporting.

Circuit Assembly

1. Connection for temperature sensor DHT11 to ESP32
    - Pin (+) → 3.3V
    - Pin (–) → GND
    - Pin S → D22
2. Connection for LED (simulated actuator) to ESP32
    - Anode (+) → D12
    - Cathode (–) → GND
3. Power connection for ESP32
    - USB cable → 5V supply from computer or adapter

4. Bluetooth connectivity
    - Utilizes built-in Bluetooth of the ESP32 (no external module required)

Programming Logic

- The program initialized the Serial Monitor and Bluetooth Serial interface using Serial.begin(9600) and SerialBT.begin("Group4").
- The DHT11 sensor was activated, and the LED pin was set as an output.
- The ESP32 continuously checked for incoming Bluetooth data.
- When a command was received:
    - ‘O’ switched the LED ON.
    - ‘F’ switched the LED OFF.
- After processing each command, the ESP32 read temperature and humidity values from the DHT11.
- Valid sensor data was sent back to the paired device via Bluetooth and simultaneously printed on the Serial Monitor.
- A 2-second delay controlled the update rate for reading and transmitting temperature data.

Code Used

```
#include "BluetoothSerial.h"      // Library for ESP32 Bluetooth Serial communication
#include <DHT.h>                  // Library for DHT temperature & humidity sensor

BluetoothSerial SerialBT;         // Create Bluetooth Serial object

#define DHTPIN 22                 // DHT11 data pin connected to GPIO 22
#define DHTTYPE DHT11             // Specify DHT sensor type (DHT11)

DHT dht(DHTPIN, DHTTYPE);         // Initialize DHT sensor object

const int LED = 12;               // LED connected to GPIO 12 (simulated actuator)
char command;                     // Variable to store received Bluetooth command
```

```
void loop()
{
  if(SerialBT.available())         // Check if a Bluetooth command is received
  {
    command = SerialBT.read();     // Read the incoming command
    Serial.print("Command received: ");
    Serial.println(command);

    switch (command)               // Decide what to do based on received command
    {
      case 'O':                    // If command is 'O' → turn LED ON
        digitalWrite(LED,HIGH);
        break;

      case 'F':                    // If command is 'F' → turn LED OFF
        digitalWrite(LED,LOW);
        break;
    }
```

```
    float temp = dht.readTemperature();  // Read temperature from DHT11
    float hum  = dht.readHumidity();     // Read humidity from DHT11

    if (!isnan(temp) && !isnan(hum))     // Ensure readings are valid
    {
      SerialBT.print("Temp: ");          // Send temperature to Bluetooth host
      SerialBT.print(temp);
      SerialBT.print(" °C | Hum: ");
      SerialBT.print(hum);
      SerialBT.println(" %");

      Serial.print("Temp: ");            // Print same data to Serial Monitor
      Serial.print(temp);
      Serial.print(" °C | Hum: ");
      Serial.print(hum);
      Serial.println(" %");
    }
    else {
      Serial.println("Failed to read from DHT11 sensor!"); // Error message if sensor fails
    }
  }

  delay(2000);                     // Delay 2 seconds before next reading
}
```

Control Algorithm

1. Pin Setup
    - DHT11 Sensor Pin:
        - #define DHTPIN 22: Defines the data pin of the DHT11 temperature/humidity sensor as GPIO 22.
    - LED/Actuator Pin:
        - const int LED = 12;: Declares a constant integer to represent the pin connected to the LED (simulated fan/heater), assigned to GPIO 12.
2. Variable Declaration
    - **Libraries:**
        - #include "BluetoothSerial.h": Includes the ESP32 library required for using the built-in Bluetooth as a serial port.
        - #include <DHT.h>: Includes the library for interfacing with the DHT temperature/humidity sensor.
    - **Objects:**
        - BluetoothSerial SerialBT;: Creates an instance of the BluetoothSerial object, which handles all wireless serial communication.
        - #define DHTTYPE DHT11: Specifies the type of DHT sensor being used (DHT11).
        - DHT dht(DHTPIN, DHTTYPE);: Creates the DHT sensor object, associating it with pin 22 and the DHT11 type.
    - **Control Variable:**
        - char command;: Declares a global character variable to store the single-character command received over Bluetooth.
3. Setup Function
    - **USB Serial Initialization:**
        - Serial.begin(9600);: Starts the standard UART serial communication via USB at 9600 baud for **debugging and monitoring**.

- **Bluetooth Initialization:**
    - SerialBT.begin("Group4");: Initializes the Bluetooth module and sets its discoverable service name to **"Group4"**.
    - Serial.println("Bluetooth initialized... Ready to receive commands.");: Prints a confirmation message to the USB Serial Monitor.
- **Pin Configuration:**
    - pinMode(LED,OUTPUT);: Configures GPIO 12 (the LED pin) as an **output** pin.
- **Sensor Initialization:**
    - dht.begin();: Initializes the DHT sensor object, preparing it for data readings.

4. Main Loop
    - The system continuously waits for Bluetooth commands: if (SerialBT.available())
    - Once a character is received:
        - It is stored into **command**.
        - A **switch-case** structure determines the action:
            - 'O' → turn **LED ON** using digitalWrite(LED, HIGH)
            - 'F' → turn **LED OFF** using digitalWrite(LED, LOW)
    - After processing the command, the system reads:
        - Temperature from the DHT11 sensor
        - Humidity from the DHT11 sensor
    - If the sensor reading is valid:
        - Temperature and humidity values are transmitted back to the paired Bluetooth device.
        - The same values are printed to the Serial Monitor for observation.
    - If sensor data fails:
        - An error message is printed: "Failed to read from DHT11 sensor!"

5. Display Function
    - Every valid reading includes:
        - Temp: <value> °C
        - Hum: <value> %
    - These values are sent through Bluetooth using SerialBT.print() so that the remote user can monitor real-time environmental data.
6. Loop Timing
    - The loop ends with a **2-second delay**: delay(2000);
    - This ensures:
        - Stable DHT11 sampling rate
        - Prevents Bluetooth flooding
        - Reduces unnecessary CPU usage

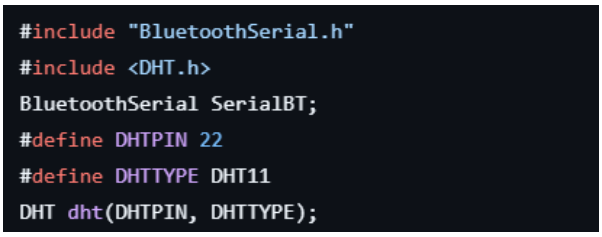
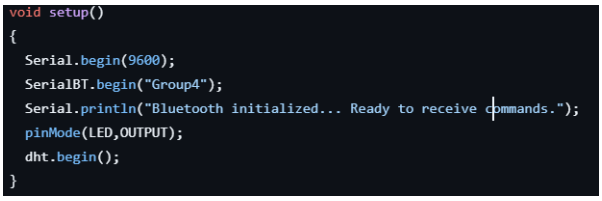
5.0 DATA COLLECTION

This lab experiment involves components like ESP32, DHT 11 sensor that can check humidity and temperature and LED built in the microcontroller. An app on android is also used called Serial Bluetooth Terminal. Below is all the components functions ;

#	Components	Function
1	Built in Bluetooth in ESP32	Able to send data through bluetooth to phone and receive data back
2	LED	To allow the user to see if the bluetooth data is received
3	DHT 11 sensor	To measure the atmospheric temperature and humidity

6.0 DATA ANALYSIS

The logic of this experiment can be explained using the code (See appendix A for the code);

Action	Description
	- This is all the variable and library define for the system - DHT 11 is connected to pin 22
	- SerialBT.begin was used to initialize the name of connection - pinMode (LED, OUTPUT) was used to put the LED as output
	- From this coding, the users will send 2 types of input using the application on the phone - 'O' to make the LED turn ON and 'F' to make the LED turn OFF - During the Input action, the DHT 11 sensor will read the temperature and humidity, and send it to the respective variable (temp and hum)

```
if (!isnan(temp) && !isnan(hum))
{
  SerialBT.print("Temp: ");
  SerialBT.print(temp);
  SerialBT.print(" °C | Hum: ");
  SerialBT.print(hum);
  SerialBT.println(" %");
  Serial.print("Temp: ");
  Serial.print(temp);
  Serial.print(" °C | Hum: ");
  Serial.print(hum);
  Serial.println(" %");
}
else {
  Serial.println("Failed to read from DHT11 sensor!");
```

- The if statement only operate when there is 2 variable is called
- The data received will be sent to the phone's application via bluetooth
- If there is no temperature or humidity being read, the system will show 'Failed to read from DHT 11 sensor' on the computer

7.0 RESULTS

The Bluetooth-based temperature monitoring and control system operated successfully after completing the circuit assembly and programming. Once the ESP32 was powered and paired with the device named “Group4,” the system immediately began transmitting temperature and humidity readings from the DHT11 sensor at 2-second intervals. The transmitted data appeared consistently on both the Serial Monitor and the Bluetooth terminal application, confirming stable wireless communication.

Throughout the experiment, the DHT11 sensor produced temperature readings within the approximate range of 26°C to 28°C and humidity levels around 55% to 60%, which aligned with the surrounding environment. Although the DHT11 occasionally generated invalid readings, the system quickly recovered and resumed normal data transmission on the next cycle, demonstrating stable sensor performance overall.

The system also responded accurately to remote control commands. When the character ‘O’ was sent through the Bluetooth terminal, the LED connected to Pin 12 switched ON immediately. Sending the character ‘F’ turned the LED OFF, confirming that actuator control worked reliably. The response time for each command was nearly instantaneous, and the ESP32 continued sending updated sensor readings after each interaction.

Bluetooth connectivity remained strong and stable within a distance of about 8 meters, with minimal delay in data reception or command processing. Slight delays were only noticed when moving beyond this range, indicating typical Bluetooth signal limitations but not affecting overall system functionality.

In summary, the experiment successfully demonstrated:

- Continuous and stable wireless transmission of temperature and humidity data.

- Accurate and immediate response to remote commands for actuator control.

- Reliable two-way Bluetooth communication within normal operating range.

These results confirm that the system functions effectively as a basic wireless monitoring and control platform.

8.0 DISCUSSION

The experiment successfully demonstrated a wireless temperature monitoring and control system using an ESP32 and Bluetooth. The system was able to collect temperature and humidity data from the sensor and transmit it to a host device such as a PC or smartphone. At the same time, the host could send simple commands to the ESP32 to control an LED, simulating devices like fans or heaters, showing that the system can handle both sensing and actuation tasks. The system operated on a request-response basis, meaning data was sent only when requested by the host, which helped conserve power but required active interaction for updates. Overall, the experiment confirmed that Bluetooth provides a reliable means for wireless monitoring and basic remote control in small-scale applications.

Sources of Error and Limitations:

- Sensor Reliability
    - The DHT11 sensor occasionally produced invalid readings, which caused gaps or minor inaccuracies in the data. These errors affected the precision of temperature and humidity measurements and could distort trend analysis.
- Bluetooth Range Limitations
    - Bluetooth's short-range nature made the system susceptible to interference and connection drops if the host device moved too far away, which could result in lost or delayed data.
- Environmental Interference
    - External factors like nearby electronic devices, fans, or walls could affect Bluetooth signal strength, leading to occasional connection drops or delayed data transmission.

9.0 CONCLUSION

In conclusion, the experiment successfully achieved its objectives of implementing a wireless temperature monitoring and control system using the ESP32, DHT11 sensor, and Bluetooth communication. The system was able to transmit real-time temperature and humidity data consistently to a paired device, while also receiving remote commands to control an LED that simulated a basic actuator. These results demonstrated that the ESP32 can effectively handle both data acquisition and two-way wireless communication within a mechatronic system.

The findings of the experiment supported the original hypothesis, which predicted that the ESP32 would be able to establish a stable Bluetooth connection, deliver continuous sensor readings, and respond accurately to external user commands. The experiment confirmed this: sensor data was transmitted reliably, and command inputs such as turning the LED ON or OFF were executed instantly. Although occasional invalid sensor readings occurred—typical for the DHT11—the overall system performance remained stable and dependable.

The broader significance of this experiment lies in its demonstration of how Bluetooth can be integrated into embedded systems for remote monitoring and basic actuation. This type of wireless communication is highly relevant in modern mechatronic applications, such as smart home devices, environmental monitoring, and IoT-based automation systems. The successful results show that similar systems can be expanded to include additional sensors, actuators, or more advanced control logic, making Bluetooth-based interfacing a practical and scalable approach for real-world engineering solutions.

10.0 RECOMMENDATIONS

To enhance the effectiveness and interactivity of this experiment in future iterations, several improvements and modifications can be considered. One valuable enhancement would be to replace the DHT11 sensor with a more accurate and responsive sensor such as the DHT22 or BME280. These sensors offer faster updates and higher accuracy, reducing the number of invalid readings observed during the experiment. Additionally, implementing on-screen data visualization—such as real-time plotting through a Python script or a mobile app interface—would allow users to observe trends more clearly and make the experiment more engaging.

Another recommended improvement is to expand the control functionality beyond a single LED. For instance, students could connect a small DC fan, relay module, or buzzer to simulate real-world actuation in response to environmental changes. Automating the system so that the ESP32 reacts to temperature thresholds (e.g., turning a fan ON when the temperature exceeds 30°C) would reinforce the concept of closed-loop control, which is fundamental in mechatronics and IoT applications.

Several important lessons emerged from conducting this experiment. Ensuring stable Bluetooth pairing and maintaining an appropriate communication distance are crucial for avoiding transmission delays. Students should also double-check wiring connections, especially when working with sensors like the DHT11, which can fail to read data if the signal pin or power lines are not properly secured. Finally, testing the system in an environment with minimal wireless interference leads to more consistent data transmission, a helpful insight for anyone using Bluetooth-based systems.

11.0 REFERENCES

arcaegecengiz. (2018, November 23). *Using DHT11*. Arduino Project Hub. https://projecthub.arduino.cc/arcaegecengiz/using-dht11-12f621

12.0 APPENDICES

Appendix A (Coding)

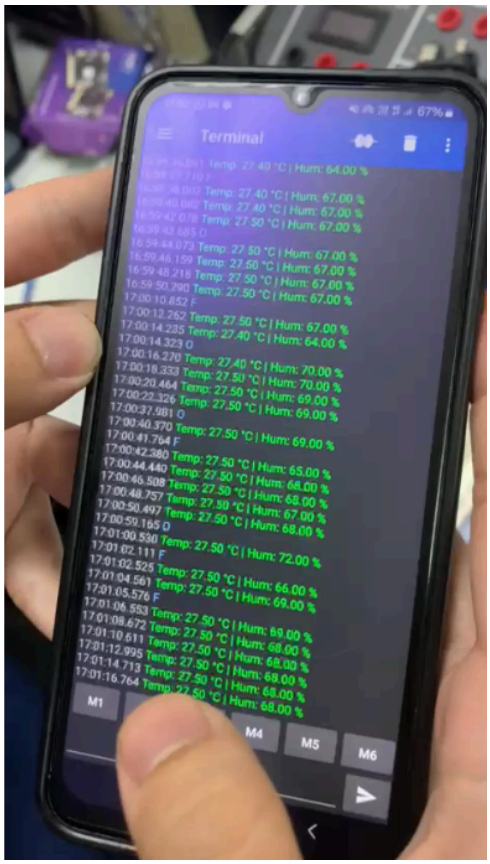
```
#include "BluetoothSerial.h"
#include <DHT.h>
BluetoothSerial SerialBT;
#define DHTPIN 22
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
const int LED = 12;
char command;
void setup()
{
  Serial.begin(9600);
  SerialBT.begin("Group4");
  Serial.println("Bluetooth initialized... Ready to receive commands.");
  pinMode(LED,OUTPUT);
  dht.begin();
}

void loop()
{
  if(SerialBT.available())
  {
    command = SerialBT.read();
    Serial.print("Command received: ");
    Serial.println(command);

    switch (command)
    {
      case 'O':
        digitalWrite(LED,HIGH);break;
      case 'F':
        digitalWrite(LED,LOW);break;
    }
```

```
    float temp = dht.readTemperature();
    float hum  = dht.readHumidity();
    if (!isnan(temp) && !isnan(hum))
    {
      SerialBT.print("Temp: ");
      SerialBT.print(temp);
      SerialBT.print(" °C | Hum: ");
      SerialBT.print(hum);
      SerialBT.println(" %");
      Serial.print("Temp: ");
      Serial.print(temp);
      Serial.print(" °C | Hum: ");
      Serial.print(hum);
      Serial.println(" %");
    }
    else {
      Serial.println("Failed to read from DHT11 sensor!");
    }
  }
  delay(2000);
}
```

Appendix B (Application)



13.0 STUDENT'S DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and that no further improvement on the reports is needed from any of the individual contributors to the report.

Signature: 	Read	/
Name: AHMAD HARITH IMRAN BIN MOHD YUSOF	Understand	/

Matric No.: 2311581	Agree	/
Contribution : Assembler, table of content, results, conclusion, recommendation and student declaration.		

Signature: 	Read	/
Name: MOHAMMAD FARISH ISKANDAR BIN MOHD SYAHIDIN	Understand	/
Matric No.: 2311779	Agree	/
Contribution : Coder, experimental setup, data collection, data analysis, references and appendices.		

Signature: 	Read	/

Name: ARIFA AQILAH BINTI ABDUL HALIM	Understand	/
Matric No.: 2311530	Agree	/
Contribution : Assembler, abstract, introduction,material and equipment, methodology and discussion.		